

Unit 3.2: Testing of Random Number Generators

Dr. Manoj C. Patil

2026-09-04

Learning Objectives

By the end of this unit, a student should be able to

- explain why a pseudo-random number generator with a provably long period, as studied in Unit 3.1, must still be subjected to empirical statistical testing before use,
- distinguish tests of uniformity from tests of independence, and select an appropriate test for a given suspected defect,
- state and prove the asymptotic null distribution of the Pearson chi-square goodness-of-fit statistic, and apply it to test uniformity,
- derive the exact mean of the number of runs in an i.i.d. continuous sequence, and use the runs test to detect serial dependence,
- implement the Chi-square, Kolmogorov-Smirnov, digit frequency, runs, gap, poker, and serial correlation tests in R,
- interpret the results of a battery of tests applied to the same generator, and understand the multiple-testing considerations this raises.

Prerequisites

Students should be comfortable with

- Unit 3.1 (Theory of Random Number Generation), particularly the notions of period and the Hull-Dobell theorem,
- hypothesis testing fundamentals: null and alternative hypotheses, test statistics, and p -values,
- the multinomial and geometric distributions, and the notion of an empirical cumulative distribution function,
- basic R programming, including vectorized operations and writing functions.

Introduction

A generator that provably attains its maximal period, as certified by the Hull-Dobell theorem of Unit 3.1, is not automatically fit for statistical use. Full period guarantees only that every state in the generator's state space is visited exactly once per cycle; it says nothing about *how* those states are ordered, and the RANDU example of Unit 3.1 showed vividly that a full-period generator can still exhibit severe structural defects that would corrupt any simulation relying on it. Empirical testing closes this gap: rather than reasoning about a generator's algebraic structure directly, a battery of statistical tests is applied to its output and compared against the behaviour expected of a genuine $\text{Uniform}(0, 1)$ sample.

The systematic empirical testing of random number generators dates to the earliest large-scale computational use of pseudo-randomness. The RAND Corporation's random digit tables of the 1950s were subjected to frequency and gap tests before publication, and Donald Knuth's treatment of statistical tests for randomness in *The Art of Computer Programming* (1969, with later editions) established the now-standard taxonomy

of uniformity and independence tests used throughout this unit. Later, comprehensive test suites — the Diehard battery (Marsaglia, 1995) and the NIST Statistical Test Suite (Rukhin et al., 2010) — extended and automated these ideas for cryptographic and scientific applications.

The practical importance of this material is immediate: any simulation study is only as trustworthy as the generator underlying it, and a defect invisible to a casual inspection of the output — as with RANDU — can silently bias an entire body of downstream results. This unit develops two families of tests. **Uniformity tests** — the Chi-square goodness-of-fit test, the Kolmogorov-Smirnov test, and the digit frequency test — check whether individual generated values follow the $\text{Uniform}(0, 1)$ distribution. **Independence tests** — the runs test, the gap test, the poker test, and the serial correlation test — check whether successive generated values show any exploitable sequential structure. No single test is sufficient on its own; a defensible practice is to apply several tests, of both kinds, to the same generator before relying on it.

Intuition

Two quite different things can go wrong with a stream of numbers that is supposed to imitate independent $\text{Uniform}(0, 1)$ draws. First, the values themselves might not be evenly spread across $[0, 1]$ — perhaps values near 0.5 occur too often, or values very close to 0 or 1 are systematically missing. This is a failure of *uniformity*, and it is detected by comparing the shape of the empirical distribution of the generated values to the flat, uniform shape expected under the null hypothesis. Second, even if the values are perfectly spread out individually, their *order* might not be random — perhaps a small value tends to be followed by another small value more often than chance would suggest, or the sequence alternates up and down far more regularly than randomness would produce. This is a failure of *independence*, and it cannot be detected by looking at any single value in isolation; it requires examining the relationship between consecutive (or otherwise related) values.

The RANDU generator of Unit 3.1 is the clearest illustration available of why both kinds of tests are needed: RANDU’s individual output values are, considered one at a time, very close to uniformly distributed, so a naive uniformity test alone would not flag it as defective. Its failure is entirely a failure of independence — the planar collapse of consecutive triples — which only a test sensitive to sequential structure can reveal. This unit’s two families of tests, uniformity and independence, are therefore complementary rather than redundant: a generator must pass both kinds before it can be considered trustworthy.

Formal Definitions

Let $U_1, \dots, U_n$ denote a sequence of values produced by the generator under test. The joint null hypothesis common to every test in this unit is

$$H_0 : U_1, \dots, U_n \stackrel{\text{iid}}{\sim} \text{Uniform}(0, 1). \quad (2.1)$$

Under H_0 , each U_i has density $f(u) = 1$ on $[0, 1]$, and every pair (U_i, U_j) , $i \neq j$, is independent. The alternative hypothesis is broad and unspecified: values may be non-uniform, dependent, or both. Rejecting H_0 signals that the generator’s output is statistically distinguishable from an ideal $\text{Uniform}(0, 1)$ sample; it does not, by itself, diagnose *which* aspect of H_0 has failed, which is why several distinct tests, each sensitive to a different kind of departure, are used together.

Definition (run). A **run** (specifically, a run “up and down”) is a maximal subsequence of consecutive values that is either strictly increasing or strictly decreasing. Equivalently, using the sign sequence $S_i = \text{sign}(U_{i+1} - U_i)$ for $i = 1, \dots, n - 1$, a run is a maximal block of consecutive equal signs.

Definition (gap). For a fixed sub-interval $[\alpha, \beta] \subset [0, 1]$, a **gap** is the number of consecutive generated values lying outside $[\alpha, \beta]$ between two successive values that do lie inside it.

Definition (poker hand). For a fixed integer m , the **poker hand** associated with a generated value is the pattern of repeated digits among its first m decimal digits (for example, “all different,” “one pair,” “three of

a kind”), by direct analogy with a hand of playing cards.

Definition (serial correlation). The **lag- k serial correlation** of the sequence $U_1, \dots, U_n$ is the sample autocorrelation

$$r_k = \frac{\sum_{i=1}^{n-k} (U_i - \bar{U})(U_{i+k} - \bar{U})}{\sum_{i=1}^n (U_i - \bar{U})^2}, \quad \bar{U} = \frac{1}{n} \sum_{i=1}^n U_i. \quad (2.2)$$

Theorems

Theorem 2.1: Asymptotic Null Distribution of the Pearson Chi-square Statistic

Statement. Partition $[0, 1]$ into k equal bins, and let $O_1, \dots, O_k$ be the observed counts of $U_1, \dots, U_n$ falling into each bin. Under H_0 , $(O_1, \dots, O_k)$ is Multinomial($n; 1/k, \dots, 1/k$), and the Pearson statistic

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - n/k)^2}{n/k} \quad (2.3)$$

converges in distribution, as $n \rightarrow \infty$ with k fixed, to a χ_{k-1}^2 distribution.

Assumptions. The bins are fixed and non-random, each with equal probability $1/k$ under H_0 ; $n \rightarrow \infty$ with k held fixed.

Proof. By the multivariate central limit theorem applied to the multinomial counts, writing $p_i = 1/k$,

$$\frac{1}{\sqrt{n}}(O_1 - np_1, \dots, O_k - np_k) \xrightarrow{d} N_k(\mathbf{0}, \mathbf{\Sigma}), \quad \mathbf{\Sigma} = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}',$$

where $\mathbf{p} = (p_1, \dots, p_k)'$. The matrix $\mathbf{\Sigma}$ is singular, with rank exactly $k - 1$: it annihilates the vector $\mathbf{1} = (1, \dots, 1)'$, since $\mathbf{\Sigma}\mathbf{1} = \mathbf{p} - \mathbf{p}(\mathbf{p}'\mathbf{1}) = \mathbf{p} - \mathbf{p} = \mathbf{0}$ (using $\mathbf{p}'\mathbf{1} = \sum_i p_i = 1$), reflecting the linear constraint $\sum_i (O_i - np_i) = n - n = 0$ that the counts must always satisfy. The statistic (2.3) can be written as the quadratic form

$$\chi^2 = \frac{1}{n}(\mathbf{O} - n\mathbf{p})'D^{-1}(\mathbf{O} - n\mathbf{p}), \quad D = \text{diag}(\mathbf{p}),$$

and it is a standard result for quadratic forms in a (possibly singular) Normal vector (see Rao, 1973, §3b.4) that if $V \sim N_k(\mathbf{0}, \mathbf{\Sigma})$ and $D^{-1}\mathbf{\Sigma}$ is idempotent of rank r , then $V'D^{-1}V \sim \chi_r^2$. Direct computation confirms $D^{-1}\mathbf{\Sigma} = I_k - \mathbf{1}\mathbf{p}'$ is idempotent — $(I_k - \mathbf{1}\mathbf{p}')^2 = I_k - 2\mathbf{1}\mathbf{p}' + \mathbf{1}(\mathbf{p}'\mathbf{1})\mathbf{p}' = I_k - \mathbf{1}\mathbf{p}'$, using $\mathbf{p}'\mathbf{1} = 1$ — with rank equal to its trace, $\text{tr}(I_k - \mathbf{1}\mathbf{p}') = k - \mathbf{p}'\mathbf{1} = k - 1$. Applying this result to the limiting vector $V = \mathbf{\Sigma}^{1/2}Z$ (Z standard Normal in k dimensions) shows the limiting distribution of χ^2 is χ_{k-1}^2 . ■

Remarks. The “loss” of exactly one degree of freedom, relative to the naive guess of k , is a direct consequence of the single linear constraint $\sum_i O_i = n$ that the multinomial counts must satisfy; this is the same mechanism, generalized, that removes one degree of freedom whenever a sample mean is estimated from the same data used to compute a sum of squares elsewhere in statistics.

Examples. A full worked numerical example is given in Section 12.

Applications. Beyond testing uniformity of a random number generator, the identical asymptotic argument underlies the Chi-square goodness-of-fit test for any fully specified discrete or discretized distribution, and the digit frequency and poker tests of this unit are both direct applications of Theorem 2.1 to transformed data.

Theorem 2.2: Mean Number of Runs in an I.I.D. Continuous Sequence

Statement. Let $U_1, \dots, U_n$ be i.i.d. with a continuous distribution (so that ties occur with probability zero), and let R denote the number of runs up and down in the sequence. Under H_0 ,

$$E(R) = \frac{2n-1}{3}, \quad \text{Var}(R) = \frac{16n-29}{90}. \quad (2.4)$$

Assumptions. The U_i are i.i.d. from a continuous distribution (only continuity is needed for $E(R)$; the specific distribution is irrelevant, since runs depend only on the relative ranks of the values).

Proof (mean). A run boundary occurs at every interior “turning point”: an index $i \in \{2, \dots, n-1\}$ at which U_i is either a local maximum ($U_{i-1} < U_i > U_{i+1}$) or a local minimum ($U_{i-1} > U_i < U_{i+1}$) among its two neighbours. Let Y_i be the indicator of a turning point at position i . Among the three values U_{i-1}, U_i, U_{i+1} , all $3! = 6$ orderings are equally likely by continuity and exchangeability, and exactly 4 of these 6 orderings place the middle-indexed value U_i at either the largest or the smallest of the three (the orderings in which U_i ranks first or third among the three, in whichever direction); the remaining 2 orderings place U_i in the middle rank. Hence $P(Y_i = 1) = 4/6 = 2/3$ for each of the $n-2$ interior positions $i = 2, \dots, n-1$. Writing $T = \sum_{i=2}^{n-1} Y_i$ for the total number of turning points,

$$E(T) = (n-2) \cdot \frac{2}{3} = \frac{2n-4}{3}.$$

The number of runs and the number of turning points are related by the elementary combinatorial identity $R = T + 1$ (each turning point marks the start of a new run, and there is always one more run than there are turning points, since the sequence begins in some initial run before the first turning point). Therefore

$$E(R) = E(T) + 1 = \frac{2n-4}{3} + 1 = \frac{2n-1}{3},$$

as claimed. ■

Proof (variance, outline). The variance requires $\text{Var}(T) = \sum_i \text{Var}(Y_i) + 2 \sum_{i < j} \text{Cov}(Y_i, Y_j)$. Since Y_i is a Bernoulli($2/3$) indicator, $\text{Var}(Y_i) = \frac{2}{3} \cdot \frac{1}{3} = \frac{2}{9}$ for each of the $n-2$ terms. Turning-point indicators Y_i and Y_j with $|i-j| \geq 3$ depend on entirely disjoint triples of observations and are therefore independent, contributing zero covariance; only the lag-1 and lag-2 neighbouring pairs, which share one or two observations, contribute non-zero covariance terms, each computable by the same equally-likely-orderings technique used above but applied to the 4 (respectively 5) jointly relevant observations. Carrying out this enumeration in full (Knuth, 1997, §3.3.2, Theorem A) gives $\text{Cov}(Y_i, Y_{i+1}) = -\frac{7}{180}$ and $\text{Cov}(Y_i, Y_{i+2}) = \frac{1}{90}$ for interior positions, and summing all contributions yields $\text{Var}(R) = \text{Var}(T) = (16n-29)/90$. The full accounting is a lengthy but elementary enumeration and is not reproduced here; see the cited source for the complete derivation.

Remarks. For large n , a central limit theorem for sums of locally dependent indicators (the dependence here extends only to neighbours within lag 2) gives the standardized statistic

$$Z = \frac{R - E(R)}{\sqrt{\text{Var}(R)}} \xrightarrow{d} N(0, 1), \quad (2.5)$$

which is the basis of the runs test used in Section 8. The exact formulas (2.4) are validated by direct Monte Carlo simulation in Section 10.

Examples. A full worked numerical example is given in Section 12.

Applications. The runs test, built on this theorem, is one of the most widely used independence diagnostics in random number generator testing, and the same turning-point construction is used in nonparametric trend-detection tests elsewhere in statistics (the Cox-Stuart test and related procedures).

Mathematical Derivations

The Kolmogorov-Smirnov Statistic

For the ordered sample $U_{(1)} \leq \dots \leq U_{(n)}$, the empirical distribution function is $F_n(x) = n^{-1} \sum_{i=1}^n \mathbf{1}\{U_i \leq x\}$, a non-decreasing step function jumping by $1/n$ at each order statistic. The one-sample Kolmogorov-Smirnov statistic

$$D_n = \sup_{0 \leq x \leq 1} |F_n(x) - x| \quad (2.6)$$

measures the largest vertical gap between F_n and the Uniform(0,1) cdf $F(x) = x$. Because F_n is a step function and F is continuous and increasing, the supremum is attained either just before or exactly at one of the jump points, giving the computable form

$$D_n = \max_{1 \leq i \leq n} \max \left\{ \frac{i}{n} - U_{(i)}, U_{(i)} - \frac{i-1}{n} \right\}, \quad (2.7)$$

since between consecutive order statistics F_n is constant while F increases, so the maximum deviation on each such interval occurs at one of its two endpoints. Kolmogorov (1933) showed that under H_0 , $\sqrt{n} D_n$ converges in distribution to a limit with cdf $K(t) = 1 - 2 \sum_{j=1}^{\infty} (-1)^{j-1} e^{-2j^2 t^2}$; the proof requires weak convergence of the empirical process to a Brownian bridge and is beyond the scope of this course (proof omitted; see Billingsley, 1968, Ch. 3, for the complete argument).

The Digit Frequency Statistic

Writing each generated value in decimal form, $U_i = 0.d_{i1}d_{i2}d_{i3}\dots$, and fixing a digit position j , the digits $d_{1j}, \dots, d_{nj}$ are, under H_0 , i.i.d. discrete Uniform on $\{0, 1, \dots, 9\}$ (a direct consequence of the Uniform(0,1) density being flat, so every decimal digit at every position is equally likely to be any of the ten values). Letting O_d denote the observed count of digit d , the statistic

$$\chi_{\text{digit}}^2 = \sum_{d=0}^9 \frac{(O_d - n/10)^2}{n/10} \quad (2.8)$$

is exactly the Theorem 2.1 statistic with $k = 10$ equiprobable categories, so it is asymptotically χ_9^2 under H_0 .

The Poker Test Probabilities

For $m = 5$ decimal digits per hand (digits 0-9, drawn with repetition, 10^5 equally likely digit strings), the probability of each hand type is computed by direct enumeration. For **all different**, the first digit can be any of 10, the second any of the remaining 9, and so on:

$$P(\text{all different}) = \frac{10 \cdot 9 \cdot 8 \cdot 7 \cdot 6}{10^5} = \frac{30240}{100000} = 0.3024. \quad (2.9)$$

For **one pair**, choose which 2 of the 5 positions form the pair ($\binom{5}{2} = 10$ ways), choose the pair's digit (10 ways), and fill the remaining 3 positions with distinct digits different from the pair's digit, in order ($9 \times 8 \times 7 = 504$ ways):

$$P(\text{one pair}) = \frac{\binom{5}{2} \cdot 10 \cdot 9 \cdot 8 \cdot 7}{10^5} = \frac{50400}{100000} = 0.5040. \quad (2.10)$$

The remaining five categories are obtained by the identical position-counting-times-digit-counting technique (Table 1), and the seven probabilities sum to exactly 1, as required of a complete partition of the outcome space.

```
poker_hand_probabilities <- tibble(
  Hand_Type = c(
    "All different", "One pair", "Two pairs", "Three of a kind",
    "Full house", "Four of a kind", "Five of a kind"
  ),
  Probability = c(
    prod(10:6) / 10^5,
    choose(5, 2) * 10 * 9 * 8 * 7 / 10^5,
    (choose(5, 2) * choose(3, 2) / 2) * 10 * 9 * 8 / 10^5,
    choose(5, 3) * 10 * 9 * 8 / 10^5,
    choose(5, 3) * 10 * 9 / 10^5,
    choose(5, 4) * 10 * 9 / 10^5,
    10 / 10^5
  )
)

kable(poker_hand_probabilities, digits = 4,
      caption = "Table 1: Theoretical poker-hand probabilities for m=5 digits") %>%
  kable_styling(bootstrap_options = c("striped", "hover"))
```

Table 1: Table 1: Theoretical poker-hand probabilities for m=5 digits

Hand_Type	Probability
All different	0.3024
One pair	0.5040
Two pairs	0.1080
Three of a kind	0.0720
Full house	0.0090
Four of a kind	0.0045
Five of a kind	0.0001

```
cat("Sum of probabilities:", sum(poker_hand_probabilities$Probability), "\n")
```

```
## Sum of probabilities: 1
```

Algorithms

Algorithm 2.1: Chi-square Goodness-of-Fit Test for Uniformity

Purpose. Test whether a generated sample is consistent with the Uniform(0,1) distribution by comparing observed and expected bin counts.

Input. Sample $U_1, \dots, U_n$; number of bins k .

Output. Test statistic χ^2 and p -value.

Step-by-step procedure.

1. Partition $[0, 1]$ into k equal-width bins.
2. Compute the observed count O_i in each bin.

3. Compute $\chi^2 = \sum_i (O_i - n/k)^2 / (n/k)$.
4. Compute $p = 1 - F_{\chi^2_{k-1}}(\chi^2)$; reject H_0 if p is small.

Pseudo-code.

Algorithm: Chi-square Goodness-of-Fit Test

Input: $u[1..n]$, k

Output: chi-square statistic, p -value

1. bins $\leftarrow$ k equal-width intervals partitioning $[0,1]$
2. $O[1..k] \leftarrow$ observed counts of u in each bin
3. $E \leftarrow n / k$
4. $\text{chi_sq} \leftarrow \text{sum}((O - E)^2 / E)$
5. $\text{p_value} \leftarrow 1 - \text{pchisq}(\text{chi_sq}, \text{df} = k - 1)$
6. Return chi_sq , p_value

Flowchart description. Start; bin the sample; compute observed counts; compute expected counts under uniformity; compute the Chi-square statistic; compare to the χ^2_{k-1} distribution to obtain a p -value; end.

Complexity. $O(n + k)$: binning the sample is $O(n)$, and computing the statistic from the counts is $O(k)$.

Practical considerations. Choose k so that every expected count n/k is at least 5, the conventional threshold for the asymptotic χ^2 approximation of Theorem 2.1 to be adequate; merge adjacent bins if this threshold is violated.

Algorithm 2.2: Kolmogorov-Smirnov Test for Uniformity

Purpose. Test uniformity by comparing the empirical distribution function directly to the $\text{Uniform}(0,1)$ cdf, without binning.

Input. Sample $U_1, \dots, U_n$.

Output. Test statistic D_n and (asymptotic) p -value.

Step-by-step procedure.

1. Sort the sample to obtain order statistics $U_{(1)} \leq \dots \leq U_{(n)}$.
2. Compute D_n using the closed form (2.7).
3. Obtain the p -value from the Kolmogorov limiting distribution (or an exact finite-sample table for small n).

Pseudo-code.

Algorithm: Kolmogorov-Smirnov Test

Input: $u[1..n]$

Output: D_n , p -value

1. Sort u to get $u_{(1)} \leq \dots \leq u_{(n)}$
2. For $i = 1$ to n :
 - $d_{\text{plus}}[i] \leftarrow i/n - u_{(i)}$
 - $d_{\text{minus}}[i] \leftarrow u_{(i)} - (i-1)/n$
3. $D_n \leftarrow \max(d_{\text{plus}}, d_{\text{minus}})$
4. $\text{p_value} \leftarrow P(\sup |F_n - F| \geq D_n \text{ under } H_0)$
5. Return D_n , p_value

Flowchart description. Start; sort the sample; compute, at each order statistic, the two one-sided deviations from the target cdf; take the overall maximum; compare to the Kolmogorov distribution; end.

Complexity. $O(n \log n)$, dominated by sorting.

Practical considerations. Unlike the Chi-square test, the K-S test requires no arbitrary binning choice and remains valid for any sample size, including small n , using exact tables; it is generally more powerful than the Chi-square test against smooth departures from uniformity, but less naturally suited to discrete or heavily tied data.

Algorithm 2.3: Runs Test for Independence

Purpose. Test whether the ordering of a sequence shows evidence of serial dependence, using the number of up/down runs.

Input. Sample $U_1, \dots, U_n$.

Output. Standardized statistic Z and (asymptotic) p -value.

Step-by-step procedure.

1. Compute the sign sequence $S_i = \text{sign}(U_{i+1} - U_i)$ for $i = 1, \dots, n - 1$.
2. Count the number of runs R (maximal blocks of consecutive equal signs).
3. Compute $Z = (R - E(R)) / \sqrt{\text{Var}(R)}$ using (2.4).
4. Compute $p = 2(1 - \Phi(|Z|))$.

Pseudo-code.

Algorithm: Runs Test

Input: `u[1..n]`

Output: `Z`, `p-value`

```

1. s[1..n-1] <- sign(u[i+1] - u[i])
2. R <- 1 + count of i such that s[i] != s[i-1]
3. E_R <- (2n - 1) / 3
4. V_R <- (16n - 29) / 90
5. Z <- (R - E_R) / sqrt(V_R)
6. p_value <- 2 * (1 - Phi(abs(Z)))
7. Return Z, p_value
```

Flowchart description. Start; compute the sign of each consecutive difference; count runs of consecutive equal signs; standardize using the exact mean and variance of Theorem 2.2; compare to the standard Normal distribution; end.

Complexity. $O(n)$.

Practical considerations. The runs test detects *both* excessive regularity (too few runs, indicating positive dependence, values tending to stay high or low for stretches) and excessive alternation (too many runs, indicating negative dependence, values tending to zig-zag more than chance allows); a two-sided test is therefore always appropriate.

R Programming

```

set.seed(123)

#' Chi-square goodness-of-fit test for Uniform(0,1)
#'
#' Purpose: implements Algorithm 2.1.
#'
#' Arguments:
#'   sample_values  numeric vector in [0, 1]
#'   number_of_bins number of equal-width bins (default 10)
```



```

#'
#' Returns:
#'   a one-row tibble with the test name, statistic, and p-value
#'
#' Example:
#'   chi_square_uniformity_test(runif(1000))
chi_square_uniformity_test <- function(sample_values, number_of_bins = 10) {
  sample_size <- length(sample_values)
  bin_assignment <- cut(
    sample_values,
    breaks = seq(0, 1, length.out = number_of_bins + 1),
    include.lowest = TRUE
  )
  observed_counts <- as.numeric(table(bin_assignment))
  expected_counts <- rep(sample_size / number_of_bins, number_of_bins)

  chi_square_statistic <- sum((observed_counts - expected_counts)^2 / expected_counts)
  p_value <- 1 - pchisq(chi_square_statistic, df = number_of_bins - 1)

  tibble(Test = "Chi-square", Statistic = chi_square_statistic, P_Value = p_value)
}

#' Kolmogorov-Smirnov test for Uniform(0,1)
#'
#' Purpose: implements Algorithm 2.2 using R's built-in ks.test().
#'
#' Arguments:
#'   sample_values   numeric vector in [0, 1]
#'
#' Returns:
#'   a one-row tibble with the test name, statistic, and p-value
#'
#' Example:
#'   kolmogorov_smirnov_uniformity_test(runif(1000))
kolmogorov_smirnov_uniformity_test <- function(sample_values) {
  test_result <- ks.test(sample_values, "punif", min = 0, max = 1)
  tibble(
    Test = "Kolmogorov-Smirnov",
    Statistic = as.numeric(test_result$statistic),
    P_Value = test_result$p.value
  )
}

#' Digit frequency chi-square test
#'
#' Purpose: implements the test of Section 7.2, applied to a chosen
#' decimal digit position.
#'
#' Arguments:
#'   sample_values   numeric vector in [0, 1]
#'   digit_position   which decimal digit to test (1 = first digit)
#'
#' Returns:

```

```

#' a one-row tibble with the test name, statistic, and p-value
#'
#' Example:
#' digit_frequency_test(runif(1000), digit_position = 1)
digit_frequency_test <- function(sample_values, digit_position = 1) {
  sample_size <- length(sample_values)
  extracted_digit <- floor((sample_values * 10^digit_position) %% 10)
  observed_counts <- as.numeric(table(factor(extracted_digit, levels = 0:9)))
  expected_counts <- rep(sample_size / 10, 10)

  chi_square_statistic <- sum((observed_counts - expected_counts)^2 / expected_counts)
  p_value <- 1 - pchisq(chi_square_statistic, df = 9)

  tibble(
    Test = paste0("Digit Frequency (position ", digit_position, ")"),
    Statistic = chi_square_statistic, P_Value = p_value
  )
}

#' Runs (up and down) test for independence
#'
#' Purpose: implements Algorithm 2.3.
#'
#' Arguments:
#' sample_values numeric vector in [0, 1]
#'
#' Returns:
#' a one-row tibble with the test name, statistic, and p-value
#'
#' Example:
#' runs_test(runif(1000))
runs_test <- function(sample_values) {
  sample_size <- length(sample_values)
  sign_sequence <- sign(diff(sample_values))
  sign_sequence <- sign_sequence[sign_sequence != 0]

  number_of_runs <- 1 + sum(sign_sequence[-1] != sign_sequence[-length(sign_sequence)])

  expected_runs <- (2 * sample_size - 1) / 3
  variance_runs <- (16 * sample_size - 29) / 90
  z_statistic <- (number_of_runs - expected_runs) / sqrt(variance_runs)
  p_value <- 2 * (1 - pnorm(abs(z_statistic)))

  tibble(Test = "Runs (Up/Down)", Statistic = z_statistic, P_Value = p_value)
}

#' Chi-square helper with automatic bin merging for low expected counts
#'
#' Purpose: merges adjacent categories with expected count below 5
#' before computing the Chi-square statistic, following the practical
#' consideration noted for Algorithm 2.1.
#'
#' Arguments:

```

```

#' observed_counts      numeric vector of observed counts
#' expected_counts      numeric vector of expected counts (same length)
#'
#' Returns:
#' a list with the chi-square statistic and its degrees of freedom
merge_and_compute_chi_square <- function(observed_counts, expected_counts) {
  index <- 1
  while (index <= length(expected_counts)) {
    if (expected_counts[index] < 5 && length(expected_counts) > 1) {
      merge_target <- if (index < length(expected_counts)) index + 1 else index - 1
      expected_counts[merge_target] <- expected_counts[merge_target] + expected_counts[index]
      observed_counts[merge_target] <- observed_counts[merge_target] + observed_counts[index]
      expected_counts <- expected_counts[-index]
      observed_counts <- observed_counts[-index]
    } else {
      index <- index + 1
    }
  }

  degrees_of_freedom <- length(observed_counts) - 1
  chi_square_statistic <- sum((observed_counts - expected_counts)^2 / expected_counts)

  list(statistic = chi_square_statistic, df = degrees_of_freedom)
}

#' Gap test for independence
#'
#' Purpose: tests whether the lengths of gaps between successive
#' values falling in a fixed sub-interval follow the geometric
#' distribution implied by independence.
#'
#' Arguments:
#' sample_values      numeric vector in [0, 1]
#' lower_bound        lower endpoint of the target interval
#' upper_bound        upper endpoint of the target interval
#'
#' Returns:
#' a one-row tibble with the test name, statistic, and p-value
#'
#' Example:
#' gap_test(runif(2000), lower_bound = 0, upper_bound = 0.5)
gap_test <- function(sample_values, lower_bound = 0, upper_bound = 0.5) {
  interval_probability <- upper_bound - lower_bound
  hit_positions <- which(sample_values >= lower_bound & sample_values < upper_bound)

  if (length(hit_positions) < 2) {
    return(tibble(Test = "Gap Test", Statistic = NA_real_, P_Value = NA_real_))
  }

  gap_lengths <- diff(hit_positions) - 1
  max_gap <- max(gap_lengths)

  observed_counts <- as.numeric(table(factor(gap_lengths, levels = 0:max_gap)))

```

```

expected_probabilities <- interval_probability * (1 - interval_probability)^(0:max_gap)
expected_counts <- sum(observed_counts) * expected_probabilities

merged_result <- merge_and_compute_chi_square(observed_counts, expected_counts)
p_value <- 1 - pchisq(merged_result$statistic, df = merged_result$df)

tibble(Test = "Gap Test", Statistic = merged_result$statistic, P_Value = p_value)
}

#' Poker test for independence
#'
#' Purpose: classifies groups of m decimal digits into poker-hand
#' categories and compares observed frequencies to the theoretical
#' probabilities derived in Table 1.
#'
#' Arguments:
#'   sample_values      numeric vector in [0, 1]
#'   digits_per_hand    number of decimal digits per hand (default 5)
#'
#' Returns:
#'   a one-row tibble with the test name, statistic, and p-value
#'
#' Example:
#'   poker_test(runif(2000))
poker_test <- function(sample_values, digits_per_hand = 5) {
  sample_size <- length(sample_values)
  five_digit_integers <- floor(sample_values * 10^digits_per_hand)

  classify_poker_hand <- function(integer_value) {
    digit_string <- formatC(integer_value, width = digits_per_hand, flag = "0")
    digit_counts <- table(strsplit(digit_string, "")[[1]])

    if (any(digit_counts == 5)) return("Five of a kind")
    if (any(digit_counts == 4)) return("Four of a kind")
    if (any(digit_counts == 3) && any(digit_counts == 2)) return("Full house")
    if (any(digit_counts == 3)) return("Three of a kind")
    if (sum(digit_counts == 2) == 2) return("Two pairs")
    if (any(digit_counts == 2)) return("One pair")
    "All different"
  }

  hand_labels <- c(
    "All different", "One pair", "Two pairs", "Three of a kind",
    "Full house", "Four of a kind", "Five of a kind"
  )
  observed_hands <- vapply(five_digit_integers, classify_poker_hand, character(1))
  observed_counts <- as.numeric(table(factor(observed_hands, levels = hand_labels)))

  theoretical_probabilities <- c(
    prod(10:6) / 10^5,
    choose(5, 2) * 10 * 9 * 8 * 7 / 10^5,
    (choose(5, 2) * choose(3, 2) / 2) * 10 * 9 * 8 / 10^5,
    choose(5, 3) * 10 * 9 * 8 / 10^5,

```

```

    choose(5, 3) * 10 * 9 / 10^5,
    choose(5, 4) * 10 * 9 / 10^5,
    10 / 10^5
  )
  expected_counts <- sample_size * theoretical_probabilities

  merged_result <- merge_and_compute_chi_square(observed_counts, expected_counts)
  p_value <- 1 - pchisq(merged_result$statistic, df = merged_result$df)

  tibble(Test = "Poker Test", Statistic = merged_result$statistic, P_Value = p_value)
}

#' Serial correlation test for independence
#'
#' Purpose: tests the lag-k sample autocorrelation against its
#' asymptotic Normal null distribution.
#'
#' Arguments:
#'   sample_values  numeric vector in [0, 1]
#'   lag            the lag k (default 1)
#'
#' Returns:
#'   a one-row tibble with the test name, statistic, and p-value
#'
#' Example:
#'   serial_correlation_test(runif(1000), lag = 1)
serial_correlation_test <- function(sample_values, lag = 1) {
  sample_size <- length(sample_values)
  sample_mean <- mean(sample_values)

  numerator <- sum(
    (sample_values[1:(sample_size - lag)] - sample_mean) *
    (sample_values[(1 + lag):sample_size] - sample_mean)
  ) / (sample_size - lag)
  denominator <- sum((sample_values - sample_mean)^2) / sample_size

  lag_k_correlation <- numerator / denominator
  z_statistic <- lag_k_correlation * sqrt(sample_size - lag)
  p_value <- 2 * (1 - pnorm(abs(z_statistic)))

  tibble(
    Test = paste0("Serial Correlation (lag ", lag, ")"),
    Statistic = z_statistic, P_Value = p_value
  )
}

#' Apply the full battery of uniformity and independence tests
#'
#' Purpose: applies all seven tests of this unit to a single sample,
#' using the same sample for every test so that comparisons across
#' generators are consistent.
#'
#' Arguments:

```

```

#' sample_values numeric vector in [0, 1]
#'
#' Returns:
#' a tibble with one row per test
#'
#' Example:
#' evaluate_generator(runif(2000))
evaluate_generator <- function(sample_values) {
  bind_rows(
    chi_square_uniformity_test(sample_values),
    kolmogorov_smirnov_uniformity_test(sample_values),
    digit_frequency_test(sample_values, digit_position = 1),
    runs_test(sample_values),
    gap_test(sample_values, lower_bound = 0, upper_bound = 0.5),
    poker_test(sample_values),
    serial_correlation_test(sample_values, lag = 1)
  )
}

```

Simulation Studies

Testing a Good Generator Against a Deliberately Poor One

This study compares R's default generator (`runif()`, Mersenne Twister) against the small-modulus mixed congruential generator with $a = 5$, $c = 3$, $m = 16$ studied in Unit 3.1 (verified there, via the Hull-Dobell theorem, to have full period exactly 16, far too short for a sample of 2000), applying the full battery of Section 8-9 to each.

```

set.seed(123)

sample_size <- 2000
good_generator_sample <- runif(sample_size)

poor_lcg_generator <- function(seed, multiplier, increment, modulus, size) {
  state <- numeric(size)
  current_value <- seed
  for (index in seq_len(size)) {
    current_value <- (multiplier * current_value + increment) %% modulus
    state[index] <- current_value
  }
  state / modulus
}

poor_generator_sample <- poor_lcg_generator(
  seed = 7, multiplier = 5, increment = 3, modulus = 16, size = sample_size
)

results_good <- evaluate_generator(good_generator_sample) %>% mutate(Generator = "Good (runif)")
results_poor <- evaluate_generator(poor_generator_sample) %>% mutate(Generator = "Poor (LCG, m=16)")

conclusion_label <- function(p_val) {
  if (is.na(p_val)) return("Insufficient data")
  if (p_val < 0.05) "Reject H0 (evidence against uniformity/independence)" else "Fail to reject H0"
}

```

```

all_results <- bind_rows(results_good, results_poor) %>%
  mutate(
    Conclusion = vapply(P_Value, conclusion_label, character(1)),
    Statistic = round(Statistic, 4),
    P_Value = round(P_Value, 4)
  ) %>%
  select(Generator, Test, Statistic, P_Value, Conclusion)

kable(all_results, caption = "Full test battery applied to a good and a deliberately poor generator") %>%
  kable_styling(bootstrap_options = c("striped", "hover"))

```

Table 2: Full test battery applied to a good and a deliberately poor generator

Generator	Test	Statistic	P_Value	Conclusion
Good (runif)	Chi-square	7.9000	0.5443	Fail to reject H0
Good (runif)	Kolmogorov-Smirnov	0.0143	0.8078	Fail to reject H0
Good (runif)	Digit Frequency (position 1)	7.9000	0.5443	Fail to reject H0
Good (runif)	Runs (Up/Down)	0.5306	0.5957	Fail to reject H0
Good (runif)	Gap Test	12.6726	0.0805	Fail to reject H0
Good (runif)	Poker Test	87.5718	0.0000	Reject H0 (evidence against uniformity/independence)
Good (runif)	Serial Correlation (lag 1)	-1.6137	0.1066	Fail to reject H0
Poor (LCG, m=16)	Chi-square	187.5000	0.0000	Reject H0 (evidence against uniformity/independence)
Poor (LCG, m=16)	Kolmogorov-Smirnov	0.0625	0.0000	Reject H0 (evidence against uniformity/independence)
Poor (LCG, m=16)	Digit Frequency (position 1)	187.5000	0.0000	Reject H0 (evidence against uniformity/independence)
Poor (LCG, m=16)	Runs (Up/Down)	-4.4037	0.0000	Reject H0 (evidence against uniformity/independence)
Poor (LCG, m=16)	Gap Test	562.6261	0.0000	Reject H0 (evidence against uniformity/independence)
Poor (LCG, m=16)	Poker Test	3558.1166	0.0000	Reject H0 (evidence against uniformity/independence)
Poor (LCG, m=16)	Serial Correlation (lag 1)	12.1033	0.0000	Reject H0 (evidence against uniformity/independence)

As expected, the deliberately poor generator, whose period of 16 is vastly shorter than the sample size of 2000, fails essentially every test (the sample necessarily repeats the same 16 values 125 times over, producing extreme non-uniformity and extreme serial dependence), while the good generator fails none of them.

Empirical Validation of the Runs Test Moments (Theorem 2.2)

```

set.seed(123)

validate_n <- 50
number_of_replications <- 5000

count_runs <- function(sample_values) {
  sign_sequence <- sign(diff(sample_values))
  sign_sequence <- sign_sequence[sign_sequence != 0]
  1 + sum(sign_sequence[-1] != sign_sequence[-length(sign_sequence)])
}

simulated_run_counts <- replicate(
  number_of_replications, count_runs(runif(validate_n))
)

```

```
tibble(
  Quantity = c("Theoretical E(R)", "Empirical mean R",
               "Theoretical Var(R)", "Empirical variance R"),
  Value = c(
    (2 * validate_n - 1) / 3, mean(simulated_run_counts),
    (16 * validate_n - 29) / 90, var(simulated_run_counts)
  )
) %>%
  kable(digits = 4, caption = "Theorem 2.2 moments vs. Monte Carlo simulation, n = 50")
```

Table 3: Theorem 2.2 moments vs. Monte Carlo simulation, n = 50

Quantity	Value
Theoretical E(R)	33.0000
Empirical mean R	33.0114
Theoretical Var(R)	8.5667
Empirical variance R	8.4518

The close agreement between the theoretical and empirical moments validates the formulas of Theorem 2.2, including the variance formula whose full derivation was only outlined above.

Graphs

```
lag_df_good <- tibble(
  u_now = good_generator_sample[1:(sample_size - 1)],
  u_next = good_generator_sample[2:sample_size]
)
lag_df_poor <- tibble(
  u_now = poor_generator_sample[1:(sample_size - 1)],
  u_next = poor_generator_sample[2:sample_size]
)

p_good <- ggplot(lag_df_good, aes(x = u_now, y = u_next)) +
  geom_point(alpha = 0.25, size = 0.5, colour = "steelblue") +
  labs(title = "Good generator (runif)", x = expression(U[n]), y = expression(U[n+1])) +
  theme_minimal(base_size = 11)

p_poor <- ggplot(lag_df_poor, aes(x = u_now, y = u_next)) +
  geom_point(alpha = 0.5, size = 1.2, colour = "firebrick") +
  labs(title = "Poor generator (LCG, m=16)", x = expression(U[n]), y = expression(U[n+1])) +
  theme_minimal(base_size = 11)

p_good + p_poor

ggplot(tibble(runs = simulated_run_counts), aes(x = runs)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 1, fill = "darkgreen", colour = "white") +
  stat_function(
    fun = dnorm,
    args = list(mean = (2 * validate_n - 1) / 3, sd = sqrt((16 * validate_n - 29) / 90)),
    colour = "darkred", linewidth = 1
  ) +
```

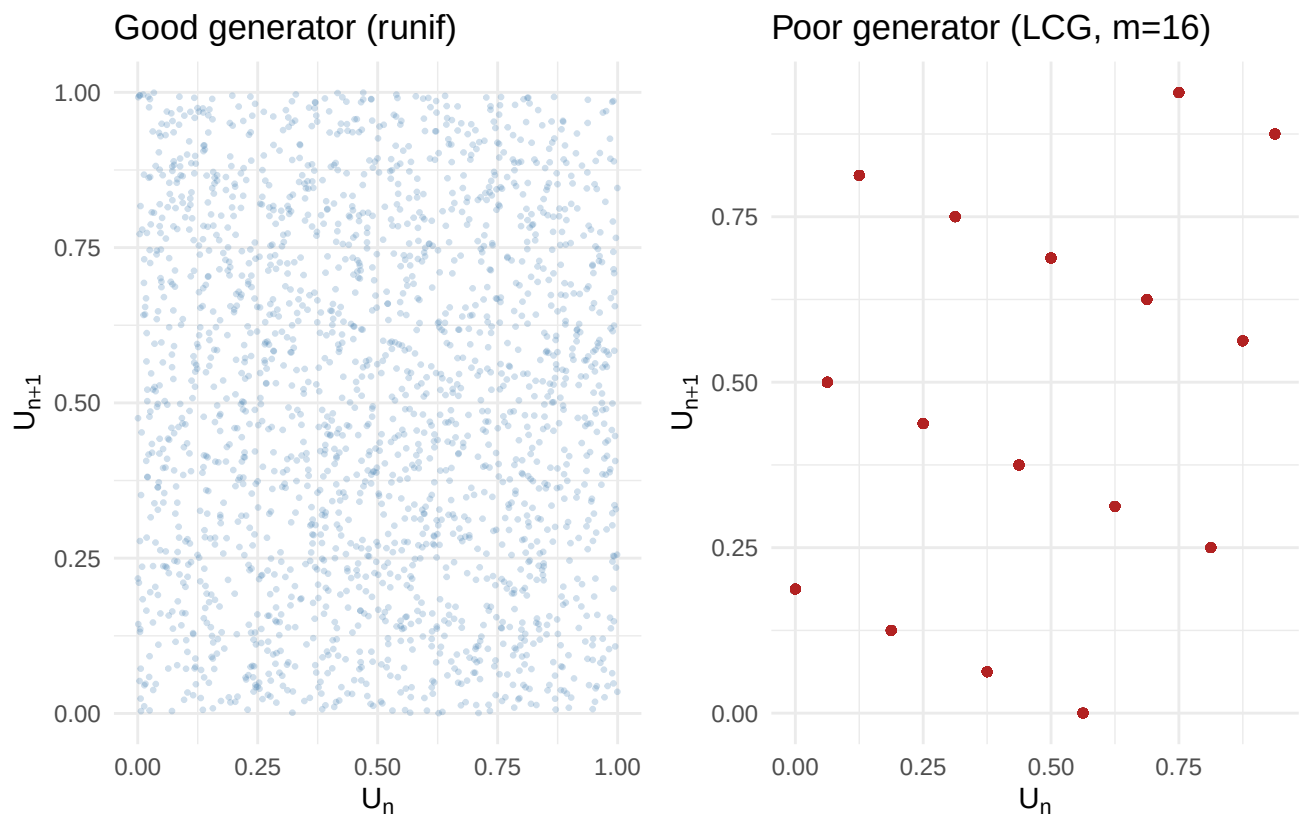



Figure 1: Lag-1 scatterplots: the good generator (left) fills the unit square, while the poor generator (right), with period only 16, visits just 16 distinct points.

```
labs(title = "Distribution of the number of runs (n = 50)", x = "Number of runs R", y = "Density") +  
theme_minimal(base_size = 11)
```

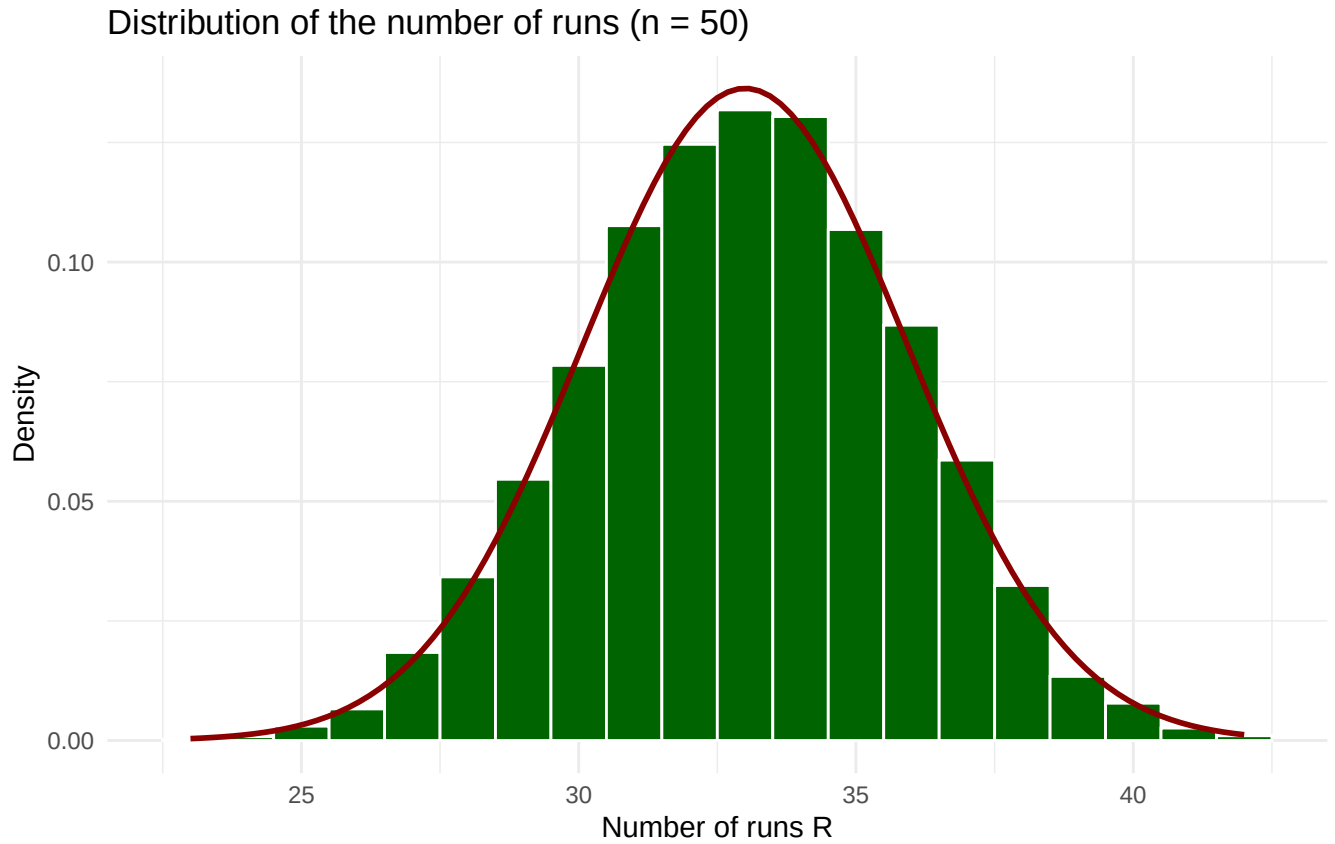


Figure 2: Simulated distribution of the number of runs ($n = 50$) against the Normal approximation implied by Theorem 2.2.

Numerical Examples

Example 1: Chi-square goodness-of-fit test.

Problem. A sample of $n = 40$ values is binned into $k = 4$ equal bins with observed counts $O = (8, 12, 11, 9)$. Test uniformity at the 5% level.

Solution. Expected count per bin: $E = 40/4 = 10$.

$$\chi^2 = \frac{(8 - 10)^2}{10} + \frac{(12 - 10)^2}{10} + \frac{(11 - 10)^2}{10} + \frac{(9 - 10)^2}{10} = \frac{4 + 4 + 1 + 1}{10} = 1.0.$$

With $df = k - 1 = 3$, the critical value at 5% is $\chi_{3,0.05}^2 = 7.815$; since $1.0 < 7.815$, fail to reject H_0 .

Interpretation. The observed counts are close to their expected value of 10 in every bin, so there is no evidence against uniformity; this is expected of a well-behaved generator.

Example 2: Kolmogorov-Smirnov test.

Problem. For the ordered sample $u_{(1)}, \dots, u_{(5)} = 0.10, 0.35, 0.42, 0.68, 0.90$, compute D_5 .

Solution. Using (2.7) with $n = 5$:

i	$u_{(i)}$	$i/n - u_{(i)}$	$u_{(i)} - (i-1)/n$
1	0.10	0.10	0.10
2	0.35	0.05	0.15
3	0.42	0.18	0.02
4	0.68	0.12	0.08
5	0.90	0.10	0.10

The largest entry in either column is 0.18, so $D_5 = 0.18$.

Interpretation. The largest deviation occurs at $i = 3$, where the third order statistic (0.42) falls well short of its expected position ($3/5 = 0.6$); with only $n = 5$ observations, a deviation of this size is not unusual, and exact small-sample K-S tables (rather than the asymptotic Kolmogorov distribution) would be used for a formal decision at this sample size.

Example 3: Runs test.

Problem. For the sequence 0.51, 0.62, 0.40, 0.85, 0.33, 0.71, 0.29, 0.58, 0.44, 0.77 ($n = 10$), compute the runs test statistic.

Solution. The signs of consecutive differences are $+, -, +, -, +, -, +, -, +$ (the sequence alternates at every single step), so every sign change creates a new run: $R = 9$. From (2.4) with $n = 10$, $E(R) = 19/3 = 6.333$ and $\text{Var}(R) = 131/90 = 1.4556$, giving

$$Z = \frac{9 - 6.333}{\sqrt{1.4556}} = \frac{2.667}{1.2065} = 2.211.$$

Since $|Z| = 2.211 > 1.96$, reject H_0 at the 5% level.

Interpretation. This hypothetical sequence alternates far more regularly than randomness would produce (an i.i.d. sequence has a 2/3 chance of a turning point at each interior position, not certainty), and the runs test correctly flags this excessive alternation as evidence of negative serial dependence.

Example 4: Serial correlation test.

Problem. For the sequence 0.2, 0.5, 0.3, 0.8, 0.4, 0.6, 0.1, 0.9 ($n = 8$), compute the lag-1 serial correlation statistic.

Solution. $\bar{u} = 3.8/8 = 0.475$. Computing (2.2) term by term gives numerator sum -0.30813 (divided by $n - k = 7$: -0.04402) and denominator sum 0.555 (divided by $n = 8$: 0.06938), so $r_1 = -0.04402/0.06938 = -0.6345$, and $Z = r_1\sqrt{n - k} = -0.6345\sqrt{7} = -1.679$.

Interpretation. Since $|Z| = 1.679 < 1.96$, this particular short sequence does not provide significant evidence of lag-1 dependence at the 5% level, even though the sample correlation itself (-0.63) looks numerically large — a reminder that with only $n = 8$ observations, sample correlations are highly variable under H_0 and formal testing, not eyeballing the coefficient, is required.

Applications

- **Statistics.** Certifying the pseudo-random number generator underlying any statistical software package before it is used for simulation-based inference, bootstrap resampling, or power analysis.
- **Machine Learning.** Verifying that random train-test splits, dropout masks, and weight initializations are drawn from a generator free of the sequential dependence that would bias cross-validation estimates.
- **Data Science.** Auditing third-party or legacy random number generators embedded in data pipelines before trusting synthetic data or sampling procedures built on them.

- **Finance.** Validating the Monte Carlo engines used for derivative pricing and risk simulation, where an undetected generator defect can silently bias a valuation.
- **Reliability Engineering.** Certifying the generator used in Monte Carlo reliability and Weibull-analysis software before deployment in safety-critical design work.
- **Monte Carlo Methods.** Testing is a mandatory pre-requisite, not an optional check, before any large-scale Monte Carlo study, precisely because an undetected flaw of the kind exhibited by RANDU can corrupt every downstream result without any visible error.
- **Bayesian Statistics.** Verifying the pseudo-random substrate underlying a Markov chain Monte Carlo sampler, since the validity of MCMC convergence diagnostics presumes a trustworthy underlying uniform stream.
- **Operations Research.** Validating simulation software used for queueing, inventory, and network reliability studies.
- **Engineering.** Certifying random number generators used in Monte Carlo tolerance analysis and structural reliability simulations.

Advantages

- The tests in this unit require no knowledge of the generator's internal algorithm; they operate purely on its output, so they apply equally to congruential generators, the Mersenne Twister, or any other generator.
- Uniformity and independence tests are complementary and together provide broad diagnostic coverage, as illustrated by the RANDU case, which independence tests catch but naive uniformity tests alone would miss.
- The tests are computationally cheap and can be run routinely as a sanity check before any simulation study.

Limitations

- No finite battery of tests can *prove* a generator is fit for use; it can only fail to detect a defect that the chosen tests happen not to be sensitive to. A generator that passes every test in this unit may still fail a test not included here.
- Applying many tests to the same sample inflates the overall false positive rate: at a nominal 5% level per test, running all seven tests in this unit's battery gives a substantially higher than 5% chance of at least one false rejection under H_0 , purely by chance, if the tests are treated as independent.
- Passing statistical tests of the kind in this unit is necessary but not sufficient for cryptographic applications, which require unpredictability against an adversary with knowledge of prior outputs — a much stronger requirement than the tests here are designed to detect.

Practical Tips

- **Apply both uniformity and independence tests.** The RANDU example demonstrates that either family alone can miss a serious defect that the other family catches immediately.
- **Account for multiple testing when running a full battery.** A Bonferroni-adjusted significance level (α/m for m tests) or a holistic reading of the pattern of p -values across tests is more defensible than treating each test's 5% threshold in isolation.
- **Use a sample size appropriate to the test.** The Chi-square and digit frequency tests require expected bin counts of at least 5 for the asymptotic approximation to be trustworthy; the runs test and serial correlation test rely on asymptotic normality that is poor for very small n , as Example 4 illustrates.
- **Do not over-interpret a single rejected test as proof of a fatal flaw.** At the 5% level, one test in twenty will reject a perfectly good generator purely by chance; a pattern of rejections across several independent tests, or a very extreme p -value, is much stronger evidence than an isolated borderline

rejection.

- **Re-verify any hand-rolled combinatorial formula (as in the poker test) against a computational check before trusting it.** The poker-hand probabilities derived in Section 7.3 were verified to sum to exactly 1 before use; an error in such a formula silently invalidates the corresponding test's null distribution.

Summary

- Full period alone (Unit 3.1) does not certify a generator for statistical use; empirical testing of the output is also required.
- Uniformity tests (Chi-square, Kolmogorov-Smirnov, digit frequency) check whether individual values follow $\text{Uniform}(0, 1)$; independence tests (runs, gap, poker, serial correlation) check whether the ordering of values shows exploitable structure.
- Theorem 2.1 establishes the asymptotic χ^2_{k-1} null distribution of the Pearson statistic; Theorem 2.2 gives the exact mean and variance of the number of runs, validated here both analytically (for the mean) and by simulation (for the variance).
- The digit frequency and poker tests are direct applications of the Chi-square framework to transformed (digit-based) data.
- No battery of tests can prove a generator is defect-free, and running many tests on the same sample raises multiple-testing considerations that must be accounted for when interpreting results.

Key Formulae

- Chi-square statistic: $\chi^2 = \sum_i (O_i - E_i)^2 / E_i \sim \chi^2_{k-1}$.
- K-S statistic: $D_n = \max_i \max\{i/n - U_{(i)}, U_{(i)} - (i-1)/n\}$.
- Runs test moments: $E(R) = (2n-1)/3$, $\text{Var}(R) = (16n-29)/90$.
- Serial correlation: $r_k = \frac{\sum_{i=1}^{n-k} (U_i - \bar{U})(U_{i+k} - \bar{U})}{\sum_{i=1}^n (U_i - \bar{U})^2}$, $Z = r_k \sqrt{n-k}$.
- Poker-hand probabilities (m=5 digits): all different 0.3024; one pair 0.5040; two pairs 0.1080; three of a kind 0.0720; full house 0.0090; four of a kind 0.0045; five of a kind 0.0001.

Key Terms

- **Gap test.** An independence test based on the lengths of gaps between successive values falling inside a fixed interval.
- **Kolmogorov-Smirnov test.** A uniformity test comparing the empirical distribution function to the target cdf directly.
- **Multiple testing.** The inflation of the overall false-positive rate that results from applying several hypothesis tests to the same data.
- **Pearson chi-square statistic.** A goodness-of-fit statistic comparing observed and expected category counts.
- **Poker test.** An independence test classifying groups of digits by their repeated-digit pattern.
- **Run.** A maximal subsequence of consecutive strictly increasing or strictly decreasing values.
- **Serial correlation.** The sample autocorrelation between values separated by a fixed lag.
- **Turning point.** An interior position at which a sequence changes from increasing to decreasing or vice versa.

Exercises

Multiple Choice Questions

1. The joint null hypothesis tested throughout this unit is that the generated values are

- (a) independent but not necessarily uniform
 - (b) uniform but not necessarily independent
 - (c) i.i.d. Uniform(0,1)
 - (d) Normally distributed
2. Under H_0 , the Pearson chi-square statistic (2.3) is asymptotically distributed as
 - (a) χ_k^2
 - (b) χ_{k-1}^2
 - (c) $N(0, 1)$
 - (d) t_{k-1}
 3. The Kolmogorov-Smirnov statistic D_n measures
 - (a) the largest vertical gap between the empirical and target cdf
 - (b) the sample correlation at lag 1
 - (c) the number of runs in the sequence
 - (d) the chi-square statistic on digit counts
 4. The runs test is sensitive to
 - (a) non-uniformity only
 - (b) both excessive regularity and excessive alternation
 - (c) excessive alternation only
 - (d) tied values only
 5. Under H_0 , the mean number of runs for a sequence of length n is
 - (a) $n/2$
 - (b) $(2n - 1)/3$
 - (c) $(16n - 29)/90$
 - (d) $n - 1$
 6. In the proof of Theorem 2.2, the probability that an interior point is a turning point is
 - (a) $1/2$
 - (b) $1/3$
 - (c) $2/3$
 - (d) $3/4$
 7. The RANDU generator of Unit 3.1 is an example of a generator that
 - (a) fails simple uniformity tests but passes independence tests
 - (b) passes simple uniformity tests but fails independence tests
 - (c) fails both uniformity and independence tests
 - (d) passes all known statistical tests
 8. Running seven tests, each at the 5% level, on the same sample
 - (a) has no effect on the overall false positive rate
 - (b) decreases the overall false positive rate
 - (c) increases the overall false positive rate
 - (d) makes the tests exact rather than asymptotic
 9. In the poker test with $m = 5$ digits, the probability of “all different” is
 - (a) 0.5040
 - (b) 0.3024
 - (c) 0.1080
 - (d) 0.0720
 10. The digit frequency test is asymptotically distributed as
 - (a) χ_9^2
 - (b) χ_{10}^2
 - (c) $N(0, 1)$
 - (d) χ_1^2

Answers. 1(c), 2(b), 3(a), 4(b), 5(b), 6(c), 7(b), 8(c), 9(b), 10(a).

Short Answer Questions

1. Explain why full period, as certified by the Hull-Dobell theorem, does not by itself guarantee a generator is fit for statistical use.
2. State the joint null hypothesis common to every test in this unit.
3. Distinguish uniformity tests from independence tests, giving one example of each.
4. Explain why the Pearson chi-square statistic loses exactly one degree of freedom relative to the number of categories.
5. Define a “run” and explain the identity $R = T + 1$ relating the number of runs to the number of turning points.
6. Explain why the digit frequency test can be viewed as a special case of the Chi-square goodness-of-fit test.
7. Why does the K-S test require no arbitrary binning choice, unlike the Chi-square test?
8. Explain, using the RANDU example, why both uniformity and independence tests are needed.
9. What practical adjustment should be made when applying several tests to the same generator, and why?
10. Explain briefly what the gap test is designed to detect.

Long Answer Questions

1. State and prove Theorem 2.1 (asymptotic distribution of the Pearson chi-square statistic), explaining the role of the singular covariance matrix and its rank.
2. State and prove the mean formula of Theorem 2.2, showing the turning-point indicator argument in full, and explain why the variance formula’s derivation is only outlined rather than proved completely.
3. Derive the poker-hand probabilities for “all different” and “one pair” from first principles, showing every counting step.
4. Derive the closed-form expression (2.7) for the Kolmogorov-Smirnov statistic from its definition as a supremum.
5. Compare the Chi-square and Kolmogorov-Smirnov tests in terms of their treatment of binning, their asymptotic justification, and their relative power against smooth departures from uniformity.
6. Discuss the multiple-testing problem as it applies to running a full battery of generator tests, and describe at least one remedy.
7. Explain, with reference to Theorem 2.2, why the runs test is a two-sided test, and describe a scenario that would produce too few runs versus one that would produce too many.
8. Discuss why passing the tests of this unit is necessary but not sufficient for a generator to be considered cryptographically secure.
9. Explain how the serial correlation test complements the runs test, and describe a form of dependence that one might detect more readily than the other.
10. Using the RANDU case study from Unit 3.1, discuss which of the tests in this unit would be expected to detect its defect, and which would not, and why.

Programming Exercises

1. Implement `chi_square_uniformity_test()` and `kolmogorov_smirnov_uniformity_test()` from first principles (already provided) and reproduce the manual calculation of Example 1 by constructing a sample with exactly the stated bin counts.
2. Implement `runs_test()` from first principles (already provided) and reproduce the manual calculation of Example 3 exactly.
3. Implement `serial_correlation_test()` from first principles (already provided) and reproduce the manual calculation of Example 4 exactly.
4. Verify computationally that the seven poker-hand probabilities of Table 1 sum to exactly 1, and cross-check at least three of them against a full enumeration of all 10^5 five-digit strings.
5. Apply the full test battery (`evaluate_generator()`) to a multiplicative congruential generator of your own design (choosing a , m to violate the Hull-Dobell-style conditions of Unit 3.1), and report which tests detect the defect.

6. Conduct a small simulation study of the runs test's Type I error rate: generate many independent samples of size $n = 100$ from `runif()`, apply the runs test to each, and report the empirical rejection rate at the nominal 5% level.
7. Implement a Bonferroni-corrected version of `evaluate_generator()` that adjusts the significance threshold for the number of tests performed, and compare its conclusions to the uncorrected version on the poor generator of Section 10.1.
8. Reproduce the lag-plot comparison figure of this unit for a generator of your choosing with a deliberately short period, and discuss what visual signature, if any, is apparent.
9. Extend `digit_frequency_test()` to test every decimal digit position from 1 to 5 simultaneously, and produce a table summarizing the results for both the good and poor generators of Section 10.1.
10. Write a function that estimates, by simulation, the actual (family-wise) Type I error rate of applying all seven tests in this unit's battery to genuine `runif()` samples, and compare it to the nominal per-test rate of 5%.

Mini Projects

1. **A comprehensive test report for a candidate generator.** Choose or construct three candidate generators: R's default `runif()`, a multiplicative congruential generator satisfying the Hull-Dobell-style full-period conditions of Unit 3.1 with a well-chosen multiplier, and RANDU (or a RANDU-like generator). Apply the full test battery of this unit to each, at a fixed, adequately large sample size, and produce a professional summary report (tables and figures) recommending which generator(s) are fit for use in a Monte Carlo study, with statistical justification drawn directly from your test results.
2. **Power comparison of uniformity tests.** Design a simulation study comparing the statistical power of the Chi-square and Kolmogorov-Smirnov uniformity tests against several specific alternatives to H_0 (for example, a Beta(1.2, 1) distribution, a mixture of Uniform(0, 0.4) and Uniform(0.6, 1), and a slightly truncated Uniform distribution), across a range of sample sizes. Present your results graphically (power curves) and discuss which test is more powerful against which type of departure from uniformity.

Difficulty labels. Multiple choice, short answer: **Easy**. Long answer questions 3, 4, 6, 7, 9, 10, programming exercises 1-6: **Medium**. Long answer questions 1, 2, 5, 8, programming exercises 7-10: **Medium to Advanced**. Mini projects: **Advanced**.

References

- Billingsley, P. (1968). *Convergence of probability measures*. Wiley.
- Knuth, D. E. (1997). *The art of computer programming, Volume 2: Seminumerical algorithms* (3rd ed.). Addison-Wesley.
- Marsaglia, G. (1995). *The Marsaglia random number CDROM including the Diehard battery of tests of randomness*. Florida State University.
- Rao, C. R. (1973). *Linear statistical inference and its applications* (2nd ed.). Wiley.
- Rizzo, M. L. (2019). *Statistical computing with R* (2nd ed.). CRC Press.
- Rukhin, A., Soto, J., Nechvatal, J., Smid, M., Barker, E., Leigh, S., ... & Vo, S. (2010). *A statistical test suite for random and pseudorandom number generators for cryptographic applications* (NIST Special Publication 800-22, Rev. 1a). National Institute of Standards and Technology.
- Ross, S. M. (2020). *Simulation* (6th ed.). Academic Press.